

Aplikacja mobilna – „Baza połowów”

Cel: zbuduj aplikację z listą połowów, dodawaniem nowych rekordów oraz zapisem danych do pliku JSON.

Dane:

- **Imie** (string)
- **Nazwisko** (string)
- **Lowisko** (string)
- **GatunekRyby** (string)
- **Waga** (double)
- **DataPolowu** (DateTime)

Punkt kontrolny 1 — Lista połowów się wyświetla

1. Utwórz projekt **.NET MAUI App**.
2. W `MainPage.xaml.cs` utwórz klasę `PolowRecord` z właściwościami: `Imie`, `Nazwisko`, `Lowisko`, `GatunekRyby`, `Waga`, `DataPolowu` (na razie zwykłe `{get; set;}`).
3. Utwórz `ObservableCollection<PolowRecord>` `Polowy` i dodaj 3 rekordy startowe, np.:
 - Jan, Kowalski, Wisła, Sandacz, 2.4, `DateTime.Now`
 - Piotr, Nowak, Odra, Szczupak, 4.1, `DateTime.Now`
 - Adam, Wiśniewski, Jezioro Nyskie, Karp, 3.7, `DateTime.Now`
4. Ustaw `BindingContext = this;`
5. W XAML dodaj `CollectionView` z `ItemsSource="{Binding Polowy}"` i pokaż w elemencie listy:
 - Imie + Nazwisko
 - Lowisko
 - GatunekRyby
 - Waga: X kg

✓ **Zaliczenie:** widoczna lista 3 połowów.

Punkt kontrolny 2 — Dodawanie nowego połowu

1. Nad listą dodaj pola `Entry` do wpisania:
 - o Imię
 - o Nazwisko
 - o Lowisko
 - o GatunekRyby
 - o Waga
2. Dodaj przycisk „Dodaj połów”.
3. W `MainPage.xaml.cs` utwórz metodę obsługującą kliknięcie przycisku.
4. W tej metodzie:
 - o pobierz dane z pól `Entry`,
 - o sprawdź, czy wszystkie pola są uzupełnione,
 - o zamień `Waga` z tekstu na liczbę,
 - o utwórz nowy obiekt `PolowRecord`,
 - o ustaw `DataPolowu = DateTime.Now`,
 - o dodaj rekord do kolekcji `Polowy`.
5. Po dodaniu rekordu wyczyść pola formularza.

✓ **Zaliczenie:** po wpisaniu danych i kliknięciu przycisku nowy połów pojawia się na liście.

Punkt kontrolny 3 — Usuwanie wszystkich połowów z listy

1. Pod formularzem albo nad listą dodaj przycisk „Usuń wszystkie rekordy”.
2. W `MainPage.xaml.cs` utwórz metodę obsługującą kliknięcie tego przycisku.
3. W tej metodzie:
 - o wyświetl okno potwierdzenia `DisplayAlert`,
 - o jeśli użytkownik wybierze **Tak**, wyczyść kolekcję `Polowy`,
 - o jeśli użytkownik wybierze **Nie**, nie rób nic.
4. Po wyczyszczeniu lista ma od razu zniknąć z ekranu.

Wskazówka:

Jeśli `CollectionView` jest podłączony do kolekcji `Polowy`, to do usunięcia wszystkich rekordów wystarczy:

```
Polowy.Clear();
```

✓ **Zaliczenie:** po kliknięciu przycisku i potwierdzeniu wszystkie rekordy znikają z listy.

Punkt kontrolny 4 — Zapis wszystkich rekordów do pliku JSON

1. Dodaj do projektu folder **Data**.
2. W folderze **Data** utwórz plik **JsonStore.cs**.
3. W pliku `JsonStore.cs` utwórz klasę `JsonStore`, która będzie odpowiadała za zapis danych do pliku JSON.
4. W klasie `JsonStore` przygotuj:
 - o pole przechowujące ścieżkę do pliku `polowy.json`,
 - o konstruktor ustawiający tę ścieżkę,
 - o metodę `SaveAllAsync(List<PolowRecord> records)`, która zapisze całą listę rekordów do pliku.
5. W `MainPage.xaml.cs` utwórz obiekt `JsonStore`.
6. W metodzie dodającej nowy połów, po dodaniu rekordu do kolekcji `Polowy`, wywołaj zapis całej kolekcji do pliku JSON.
7. Na tym etapie aplikacja ma **zapisywać dane do pliku**, ale jeszcze nie musi ich odczytywać po ponownym uruchomieniu.

Wskazówka:

W pliku `JsonStore.cs` przydadzą się:

```
using System.Text.Json;
```

oraz konstruktor i metoda podobne do:

```
private readonly string _filePath;

public JsonStore()
{
    _filePath = Path.Combine(FileSystem.AppDataDirectory, "polowy.json");
}

public async Task SaveAllAsync(List<PolowRecord> records)
{
    using var stream = File.Create(_filePath);
    await JsonSerializer.SerializeAsync(stream, records);
}
```

Po dodaniu rekordu do kolekcji możesz wywołać zapis w stylu:

```
await _store.SaveAllAsync(Polowy.ToList());
```

✔ **Zaliczenie:** po dodaniu nowego połowu rekord pojawia się na liście i cała kolekcja zapisuje się do pliku `polowy.json`.

Wyjaśnienie nowych elementów z punktu 4:

- `using System.Text.Json;`
Ta linijka dołącza narzędzia potrzebne do pracy z plikami JSON.
Dzięki temu można używać `JsonSerializer`.
- `using BazaPolowow.Models;`
Ta linijka pozwala korzystać z klasy `PolowRecord` zapisanej w folderze `Models`.
- `public class JsonStore`
To nowa klasa odpowiedzialna tylko za zapis danych do pliku JSON.
Dzięki temu kod od zapisu nie miesza się z kodem strony `MainPage`.
- `private readonly string _filePath;`
To pole przechowuje ścieżkę do pliku.
`private` oznacza, że korzysta z niego tylko klasa `JsonStore`.
`readonly` oznacza, że ścieżka ma być ustawiona raz, najczęściej w konstruktorze.
- `public JsonStore()`
To konstruktor klasy `JsonStore`.
Uruchamia się wtedy, gdy tworzymy nowy obiekt tej klasy.
- `_filePath = Path.Combine(FileSystem.AppDataDirectory, "polowy.json");`
Ta linijka ustawia pełną ścieżkę do pliku `polowy.json`.
`FileSystem.AppDataDirectory` wskazuje folder danych aplikacji.
`Path.Combine(...)` łączy folder i nazwę pliku w jedną poprawną ścieżkę.
- `public async Task SaveAllAsync(List<PolowRecord> records)`
To metoda zapisująca wszystkie rekordy do pliku.
`async` oznacza działanie asynchroniczne.
`Task` oznacza, że metoda wykonuje pracę w czasie.
`List<PolowRecord> records` oznacza, że metoda dostaje listę rekordów połowów.
- `using var stream = File.Create(_filePath);`
Ta linijka tworzy plik pod wskazaną ścieżką.
Jeśli plik już istnieje, zostanie nadpisany.
`stream` to strumień danych używany do zapisu pliku.
- `await JsonSerializer.SerializeAsync(stream, records);`
Ta instrukcja zapisuje listę `records` do pliku w formacie JSON.
`Serialize` oznacza:
zamień obiekty programu na tekst JSON i zapisz do pliku.
- `private JsonStore _store = new JsonStore();`
Ta linijka w `MainPage.xaml.cs` tworzy obiekt klasy `JsonStore`, żeby strona mogła korzystać z mechanizmu zapisu.
- `await _store.SaveAllAsync(Polowy.ToList());`
Ta linijka zapisuje całą aktualną kolekcję `Polowy` do pliku JSON.
`ToList()` zamienia `ObservableCollection` na `List`, bo tego oczekuje metoda `SaveAllAsync`.

Punkt kontrolny 5 — Odczyt rekordów z pliku JSON po uruchomieniu aplikacji

1. W pliku `JsonStore.cs` dopisz metodę `GetRecordsAsync()`, która odczyta dane z pliku `polowy.json` i zwróci listę rekordów.
2. Jeśli plik jeszcze nie istnieje, metoda ma zwrócić pustą listę.
3. W `MainPage.xaml.cs` wykorzystaj metodę uruchamianą przy wejściu na stronę, aby wczytać dane z pliku JSON.
4. Po odczycie danych:
 - wyczyść kolekcję `Polowy`,
 - dodaj do niej rekordy odczytane z pliku.
5. Dzięki temu po ponownym uruchomieniu aplikacji zapisane wcześniej rekordy znów pojawią się na liście.

Wskazówka 1 — metoda w `JsonStore.cs`:

```
public async Task<List<PolowRecord>> GetRecordsAsync()
{
    if (!File.Exists(_filePath))
        return new List<PolowRecord>();

    using var stream = File.OpenRead(_filePath);
    var data = await
        JsonSerializer.DeserializeAsync<List<PolowRecord>>(stream);
    return data ?? new List<PolowRecord>();
}
```

Wskazówka 2 — wczytanie danych w `MainPage.xaml.cs`:

```
protected override async void OnAppearing()
{
    base.OnAppearing();

    var records = await _store.GetRecordsAsync();
    Polowy.Clear();

    foreach (var record in records)
    {
        Polowy.Add(record);
    }
}
```

✓ **Zaliczenie:** po zamknięciu i ponownym uruchomieniu aplikacji wcześniej zapisane rekordy nadal są widoczne na liście.

Wyjaśnienie nowych elementów z punktu 5:

- `public async Task<List<PolowRecord>> GetRecordsAsync()`
To nagłówek metody.
`async` oznacza, że metoda działa asynchronicznie.
`Task<List<PolowRecord>>` oznacza, że metoda zwróci listę rekordów połowów.
- `if (!File.Exists(_filePath))`
Ten warunek sprawdza, czy plik `polowy.json` istnieje.
Znak `!` oznacza „nie”, więc cały zapis znaczy:
jeśli plik nie istnieje.
- `return new List<PolowRecord>();`
Jeśli pliku jeszcze nie ma, aplikacja nie może nic odczytać, więc zwraca pustą listę.
- `using var stream = File.OpenRead(_filePath);`
Ta linijka otwiera plik do odczytu.
`stream` to strumień danych, czyli sposób „czytania” zawartości pliku.
- `await JsonSerializer.DeserializeAsync<List<PolowRecord>>(stream);`
Ta instrukcja odczytuje dane z pliku JSON i zamienia je na listę obiektów `PolowRecord`.
`Deserialize` oznacza: **zamień tekst JSON na obiekty programu.**
- `return data ?? new List<PolowRecord>();`
Jeśli w zmiennej `data` są poprawnie odczytane dane, metoda je zwraca.
Jeśli `data` okaże się `null`, zwracana jest pusta lista.
Operator `??` oznacza:
jeśli lewa strona jest pusta, użyj prawej strony.
- `protected override async void OnAppearing()`
To metoda strony MAUI, która uruchamia się wtedy, gdy strona pojawia się na ekranie.
Dzięki temu można wczytać dane zaraz po otwarciu aplikacji.
- `base.OnAppearing();`
Wywołuje podstawowe działanie tej metody z klasy nadrzędnej.
To standardowy element przy nadpisywaniu tej metody.
- `var records = await _store.GetRecordsAsync();`
Ta linijka wywołuje metodę odczytu z pliku JSON i zapisuje wynik do zmiennej `records`.
- `Polowy.Clear();`
Czyści kolekcję `Polowy`, żeby nie dublować rekordów.
- `foreach (var record in records)`
Pętla przechodzi po wszystkich rekordach odczytanych z pliku, jeden po drugim.
- `Polowy.Add(record);`
Dodaje każdy odczytany rekord do kolekcji `Polowy`, a więc także do listy widocznej w aplikacji.